SAARTHI: An AI-Powered Industrial Safety Copilot for Real-Time Excavator Hazard Detection

Sounak Chowdhury, Member 2, Member 3, Member 4, Member 5

***Abstract*—Write the abstract here.**

***Index Terms*—Industrial Safety, Computer Vision, YOLOv8, Excavator, Hazard Detection, Edge AI, Large Language Models**

I. Introduction

Write the introduction here.

II. Related Work

Securing heavy-machinery operations sits at the intersection of computer vision, physiological monitoring, and adaptive decision-making. Broader surveys of industrial safety still tend to focus on rule-based alarms and single-sensor approaches, and even where hybrid deep learning architectures have made real progress on driver drowsiness detection in automotive settings, they are built around a single modality. That assumption breaks down on a construction site, a mine, or a factory floor, where operators differ widely in experience and ability and hazards rarely show up one at a time. This section works through that prior literature and situates the design choices behind SAARTHI against it.

A. Threshold-Based and Infrastructure-Centric Safety Systems

Most collision-prevention systems on active worksites still come down to proximity alarms and geofencing: fixed ultrasonic or radar sensors mounted on the machine, scanning continuously and comparing whatever they detect against a static, manually set threshold, with no regard for who's actually operating the machine. In practice, those fixed thresholds tend to misfire constantly on busy sites, where workers and equipment are always moving through the sensor's range. Worse, the same threshold gets applied to every operator regardless of experience or physical ability, so an experienced operator ends up tuning out the alarms while a newer one may not get warned early enough.

Newer cabin-integrated systems have brought computer vision into the mix, using object detection to pick out workers, vehicles, and structural hazards around the machine, and drawing on signals like facial landmark drift, eyelid closure, or chassis tilt to flag unsafe states. But most of these still work in isolation. A camera watching for fatigue has no idea there's a person standing in the machine's blind spot, and a proximity sensor has no way of knowing the operator is falling asleep. Because these hazards arise from genuinely different physical processes, no single detector is well positioned to catch all of them—which is really the argument for fusing several lightweight signals rather than trying to build one model that does everything.

Conventional alarm-based systems, built for fairly controlled, homogeneous environments, don't hold up well once the operator population and the worksite itself become this varied.

- **Rigidity and operator homogeneity.** A fixed-threshold system treats every operator identically, regardless of experience, sensory ability, or fatigue tolerance. In practice that means the same threshold that produces alarm fatigue in a veteran operator will under-warn someone newer to the job—there's no way to individualize the safeguard without changing the underlying logic entirely.
- **Deployment complexity and accessibility gaps.** Combining vision, physiological, and environmental sensing into one cabin system is already a nontrivial integration problem, and most existing architectures weren't designed with that convergence in mind. Accessibility tends to be an afterthought too: operators with hearing or visual impairments are rarely considered when alert delivery is designed almost exclusively around audio cues.

B. Fatigue-Centric and Vision-Based Defenses

A separate line of work moves detection logic onto the operator's own physiological state rather than the surrounding environment. Here the typical approach is a camera-based agent watching the driver's face—tracking eye-aspect-ratio, yawning frequency, head pose—to infer drowsiness before it becomes dangerous.

These systems are reasonably good at what they do, but that's also their limitation: they know almost nothing about what's happening outside the cabin. A fatigue detector can't tell you there's a worker nearby, or that the terrain has become unstable, or that the machine itself is tilting toward a rollover. It's built to catch one kind of hazard and stays blind to the rest.

- **Modality opacity.** Vision-only systems generally ignore machine-state telemetry altogether, even though rollover risk from chassis inclination is well documented in the tilt-sensing literature. A fatigue model has no access to that data by design, so it simply can't detect a compound hazard—drowsiness plus instability, say—even when both are present at once.
- **The context gap.** These tools are built around a single-hazard focus and stay that way in deployment. They fire only on drowsiness cues, missing nearby obstacles or

unstable ground entirely. By the time a purely fatigue-driven alert goes off, the operator may already be in a situation involving more than just tiredness.

C. Machine Learning in Industrial Safety

As static, rule-based systems have repeatedly failed to keep up with a workforce this varied in experience and accessibility needs, machine learning has become the default approach for adaptive intervention.

1) Deep Learning and Object Detection Models: Deep learning dominates recent work in this space. YOLO-based detectors, for instance, perform well in real time for identifying blind-spot obstacles and nearby workers. On the physiological side, convolutional facial-landmark models track eye and head movement continuously to catch fatigue before it becomes critical, and gradient-boosted models like XGBoost have proven useful for context-aware risk scoring, weighing environmental and operational variables against historical incident records.

The catch is that these models are almost always deployed as separate pipelines. Nobody is fusing their outputs into one coherent decision—which leaves a human supervisor to piece together several unrelated alerts in real time.

- **Fragmented decision-making.** Running fatigue, obstacle, and tilt detection as three independent systems means a supervisor has to mentally combine uncorrelated alerts on the fly. That adds real cognitive load, slows reaction time, and raises the odds that a compound hazard slips through unnoticed.
- **Static intervention policy.** Even where detection is sophisticated, the response usually isn't: cabins tend to issue the same alert regardless of context, which is exactly the kind of rigid logic an adaptive, constraint-respecting policy is meant to replace.

2) Lightweight Rule-Based Models: Some practitioners have gone the other direction, relying on fixed-weight scoring rules or manually curated alert matrices instead. These are cheap to implement and integrate easily into legacy cabin electronics, which is a real advantage.

The tradeoff is that manual thresholds don't adapt well to individual variation—different operator experience levels, different accessibility needs, changing site conditions. Keeping thresholds current across a large fleet turns into an ongoing manual calibration problem, which doesn't scale. Reinforcement learning offers a way around this, letting a system learn its own intervention policy while staying inside predefined safety bounds, though getting RL to behave safely and explainably in a live industrial setting is still an open problem—earlier work has shown that unconstrained policy learners can behave unpredictably during exploration, especially when safety constraints aren't built into training from the start.

D. The Gap Analysis

Taken together, the literature points to a fairly clear gap: nobody has really combined multiple hazard-detection modalities, adaptive decision-making, explainability, and operator-specific accessibility into one system. Most work picks one of these and pushes it further, while the others—particularly personalization—stay largely unaddressed.

SAARTHI is built around closing that specific gap. It fuses fatigue, blind-spot, tilt, and contextual risk signals into a single Reinforcement Learning policy operating under safety constraints, giving it the kind of adaptive intervention selection that has mostly been left to human judgment after the fact, while keeping the decision process explainable enough for safety-critical use. Rather than treating accessibility as something bolted on afterward, operator-specific delivery preferences are built directly into the Hybrid Decision Engine—so the same underlying risk score can reach an operator through haptic, visual, or audio channels depending on what they actually need, without changing the safety guarantees behind the decision itself.

III. System Model and Theoretical Framework

To ground the design in real operating conditions, we first characterize the driver and the machine's surrounding environment as they evolve in real time. On-device deployment inside a moving vehicle comes with hard limits—a tight latency budget and modest compute—and these constraints shape everything that follows. They rule out streaming raw, uncompressed HD frames, and they rule out running heavy spatio-temporal deep networks directly at the edge, which is exactly where traditional vehicle-tracking pipelines fall short.

SAARTHI takes a different route. It represents the operator and machine state through localized vision primitives paired with synchronized telemetry. This approach compresses the high-frequency video and sensor streams into a compact risk feature vector while preserving the information that actually matters for hazard detection: the signatures of micro-sleep, blind-spot intrusions, and axis tilt. The sections below lay out the hazard threat model, the reasoning behind real-time risk estimation, and the localized telemetry features used to flag imminent operational anomalies.

TABLE I
Comparative Analysis of Operator Safety Paradigms

Methodology	Hardware Class	Latency	Primary Limitation
Cloud-Based Telematics	Server + Cellular Node	High	Connectivity reliance in remote sites (e.g. mines)
Wearable Biosensors	Smartwatch / EEG Cap	Low	Operator discomfort, hygiene, compliance drop-off
Heavy Deep Learning (CNN/ViT)	Edge GPU (Jetson/TPU)	Medium	High computational complexity, thermal output
Basic Machine Thresholds	Machine ECU	Low	Rigidly generic, no behavioral insight
SAARTHI (Proposed)	Commodity Edge CPU	Low	Requires 30 s initial operator calibration

A. The Hazard Model

Hazards in industrial vehicle environments fall into two broad classes, separated by how visible they are and how much computational reasoning reliable detection demands.

1) Type 1: Overt Operational Hazards: A Type 1 hazard occurs whenever a physical safety boundary or operational constraint is breached—a pedestrian stepping into a blind spot, an operator letting go of the wheel with both hands, or any incursion into a predefined safety zone. Like conventional rule-based anomaly detection, these events carry clear, observable visual signatures, which makes them deterministic and comparatively straightforward to catch.

SAARTHI detects them in real time by checking the bounding boxes produced by the YOLO detector against predefined restricted regions. The moment an object crosses into a prohibited zone, the system flags a Type 1 hazard and triggers the appropriate safety response.

2) Type 2: Covert Micro-behavioral Hazards: Type 2 hazards are subtle behavioural or contextual anomalies, and they are far harder to spot. Because they tend to blend into normal operation, simple spatial rules or threshold-based vision cannot isolate them. An operator in the early stages of fatigue, for instance, may sit with an apparently normal posture and open eyes while slowly drifting into reduced alertness. Machine vibration or tilt behaves the same way: each reading may stay within acceptable bounds on its own, yet together the readings point to an abnormal operating state.

Since these situations look almost identical to a safe state on the surface, vision alone is not enough. SAARTHI instead examines the statistical character and temporal evolution of the operator's micro-movements—blink-rate variation, gaze drift, head pose—alongside machine telemetry over time. This deeper behavioural view surfaces latent hazards before they escalate into critical incidents.

B. Anomaly Theory in the Physiological and Kinematic Sense

We monitor the operator's state in real time using facial-landmark coordinate features. The core primitive is the Eye Aspect Ratio (EAR), a per-frame measure of how open the eyes are:

$$EAR = \frac{||p_2 - p_6|| + ||p_3 - p_5||}{2||p_1 - p_4||} \tag{1}$$

Here p_i denotes the 2D position of an eye landmark.

For an alert operator, the EAR sequence stays fairly stable around an open-eye mean μ_{alert}, and the fluctuations that do occur—ordinary physiological blinks, small head-pose shifts, minor lighting changes—sit within a natural range. The resulting eye-state distribution is therefore roughly bell-shaped and symmetric.

The framework rests on a simple premise: the onset of fatigue introduces specific, non-Gaussian distortions into this physiological stream.

- **Microsleep artifacts.** Prolonged eye closures—the hallmark of severe drowsiness—extend well beyond the normal blink window. They appear as rapid collapses in the EAR stream, the phenomenon the PERCLOS metric is designed to quantify, and they look nothing like the smooth, brief dip of an ordinary blink.
- **Heavy-tailed distributions.** These long closures skew the eye-state distribution and thicken its lower tail. As the operator jerks awake and tries to recover the open-eye mean μ, the distribution no longer looks symmetric—its heavier tail is the statistical fingerprint of fatigue.

Kinematic anomalies—rollover risk, blind-spot intrusion—are handled on a separate track, driven by machine telemetry and spatial bounding boxes. When the machine is moving ($v \neq 0$) and the tilt angle θ exceeds a critical set-point θ_{crit}, the response is deterministic rather than behavioural: a Tier-1 safety intervention fires immediately, independent of the standard behavioural policies.

C. Mathematical Formalization of Features

To turn these physiological cues into something a classifier can act on, the system extracts a feature vector $\mathbf{x}$ from a sliding temporal window $W = \{e_1, e_2, \ldots, e_w\}$ of raw EAR samples. Table II lists the notation used throughout the derivation.

TABLE II
MATHEMATICAL NOTATIONS AND DEFINITIONS. *These symbols define the statistical moments used to construct the feature vector* $\mathbf{x}$*, mapping raw physiological data into the decision space.*

Symbol	Definition
W	Sliding temporal window of size w
e_i	Individual EAR sample at frame i
μ	Mean EAR (first moment)
σ	Standard deviation (second moment)
S_k	Skewness (third standardized moment)
K_u	Excess kurtosis (fourth standardized moment)
E_{roc}	EAR rate of change
$\Phi(\mathbf{x})$	Non-linear mapping to the XGBoost feature space

1) Skewness (S_k): Skewness measures the asymmetry of the signal's distribution. For an alert operator the eye-openness distribution stays close to symmetric over time ($S_k \approx 0$). As fatigue sets in, micro-sleeps and long closures pull the EAR values toward the low, closed-eye end, producing an asymmetric lower tail.

$$S_k = \frac{\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^3}{\left(\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^2\right)^{3/2}} \tag{2}$$

2) Kurtosis (K_u): Kurtosis is the framework's primary discriminator for deep drowsiness. It measures the tailedness of the distribution:

$$K_u = \frac{\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^4}{\left(\frac{1}{w}\sum_{i=1}^{w}(e_i - \mu)^2\right)^2} - 3 \tag{3}$$

Subtracting 3 yields excess kurtosis, normalized against a standard normal distribution. Values far from 0 flag the kind of outliers consistent with microsleep episodes rather than ordinary blinking.

3) EAR Rate of Change (E_{roc}): The EAR rate of change captures the fast transitions typical of a fatigue episode—a sudden jerk awake, a rapid head drop:

$$|E_{roc}| = |e_t - e_{t-k}| \tag{4}$$

Here $k = 5$ sets the stride, measuring displacement across several frames. A large $|E_{roc}|$ signals a rapid eye snap or an abrupt physiological shift—motion you rarely see in calm, alert operation, where the state drifts slowly.

TABLE III
SENSITIVITY ANALYSIS OF THE FEATURES. *Each feature maps to a particular type of anomaly. Mean EAR supplies physiological background, while kurtosis and skewness pick up the non-Gaussian behaviour that characterizes the onset of fatigue.*

Feature	Physiological Meaning	Detection Function
Mean EAR (μ)	Baseline openness	Establishes the EAR baseline, providing physiological context.
Std. dev. (σ)	Blink volatility	Stability check: high variance flags erratic blinking or an unstable head pose.
Kurtosis (K_u)	Drowsiness burstiness	Primary discriminator: detects deep micro-sleeps; longer closures raise kurtosis.
Skewness (S_k)	Distribution symmetry	Fatigue artifacts: detects the closed-eye asymmetry in the distribution.
Rate of change (E_{roc})	Physiological velocity	Flags rapid eye movements or sudden head drops.

D. Defining "Real-Time" for Edge AI Safety

One of the first and crucial steps in Edge AI Safety is to clearly define what is meant by "Real-Time". However, the statistical physiological analysis requires a minimum number of w frames to be able to extract a meaningful distribution to detect fatigue.

This paper defines Real-Time in the context of Operator Safety as:

$$T_{\text{inference}} \ll T_{\text{feature\_window}} < T_{\text{hazard\_onset}} \tag{5}$$

The data collection time ($T_{\text{feature\_window}}$) is not the critical metric, but rather the Inference Latency ($T_{\text{inference}}$) is the critical metric for Real-Time in the context of Operator Safety. The continuous feature extraction window is implemented with a moving buffer. When a Critical Fatigue event occurs, or when an operator shows signs of Critical Fatigue, a verdict should be available from the Hybrid Decision Engine ($T_{\text{hazard\_onset}}$). With a considerably lower algorithmic processing time for the inference (milliseconds) compared to the time it takes the human to react in order to prevent an accident (T_{human}), the system can issue a customised warning earlier. To avoid this decision logic becoming the performance obstacle, SAARTHI will reduce $T_{\text{inference}}$ by using lightweight models optimized for the edge.

IV. PROPOSED METHODOLOGY: THE SAARTHI FRAMEWORK

SAARTHI runs on a hybrid detection engine built to make the most of the scarce compute cycles available on an in-cab edge node. The architecture divides into four logical stages, which we call Zones (Fig. 1); within Zone 3, detection splits further into two parallel tracks, Track A and Track B.

- **Zone 1 — Operating Environment:** The physical threat landscape: the overt (Type 1) and covert (Type 2) hazards present in and around the heavy machine.
- **Zone 2 — Edge Acquisition Node:** Raw data ingestion through the cabin-facing camera and the machine's telemetry interface (e.g. the CAN bus).
- **Zone 3 — Hybrid Decision Engine:** The core computational layer, housing Track A (deterministic spatial/kinematic rules) and Track B (XGBoost physiological anomaly scoring).
- **Zone 4 — Verdict & Intervention:** The final decision layer, which outputs the real-time safety status and manages the appropriate Tier-1 or Tier-2 response.

End to end, the pipeline runs in four steps:

1) **Ingestion.** Raw video frames and machine telemetry are captured and synchronized.
2) **Buffering.** EAR values and machine states accumulate in a size-w circular buffer, forming temporal sliding windows.
3) **Filtration (Track A).** Visual bounding boxes and kinematic telemetry (speed, tilt) are checked against deterministic safety constraints such as restricted-zone violations.
4) **Verification (Track B).** If Track A raises no overt hazard, the statistical physiological features (kurtosis, skewness, rate of change) are computed and passed to the XGBoost model for covert fatigue scoring.

Splitting the cheap work from the expensive work is the whole point of this design. By keeping deterministic spatial and kinematic checks (Track A) separate from the heavier statistical physiological analysis (Track B), the system spends its machine-learning budget only where it is needed. The result is a lower computational load on the edge device, which keeps thermal throttling at bay and preserves real-time responsiveness without sacrificing diagnostic precision.

A. Data Acquisition and Adaptive Preprocessing

The data flow starts in Zone 2, where the cabin-facing camera and the telemetry bus run continuously. This passive style of monitoring reflects the non-intrusive philosophy that heavy machinery demands: it sidesteps the "observer effect," since the operator is never asked to check in and is never pulled away from a delicate task. And rather than processing every frame indiscriminately, as a standard vision pipeline would, the SAARTHI node selectively intercepts and synchronizes facial landmarks with CAN bus telemetry.

Because this multi-modal stream arrives fast and the device's SRAM is limited, the system buffers it in a circular queue. The buffer acts as a shock absorber, decoupling the irregular rate at which vision primitives are extracted from the steady clock of the downstream inference engine.

1) Circular Buffer Management: Algorithm 1 formalizes how the stream is managed. Only valid frames — those with an intact face detection and high-confidence landmarks — are admitted. To avoid memory fragmentation, a persistent headache on embedded targets, the buffer allocates its memory statically and overwrites in place, always favouring the most recent temporal window.

2) Feature Engineering and Statistical Standardization: Once the temporal buffer B_t holds a full window of valid samples, the system moves into feature engineering. The main challenge here is taming the stochastic variance in the physiological and kinematic readings, so that the feature vector $\mathbf{x}$ handed to the classifier is stable enough to trust.

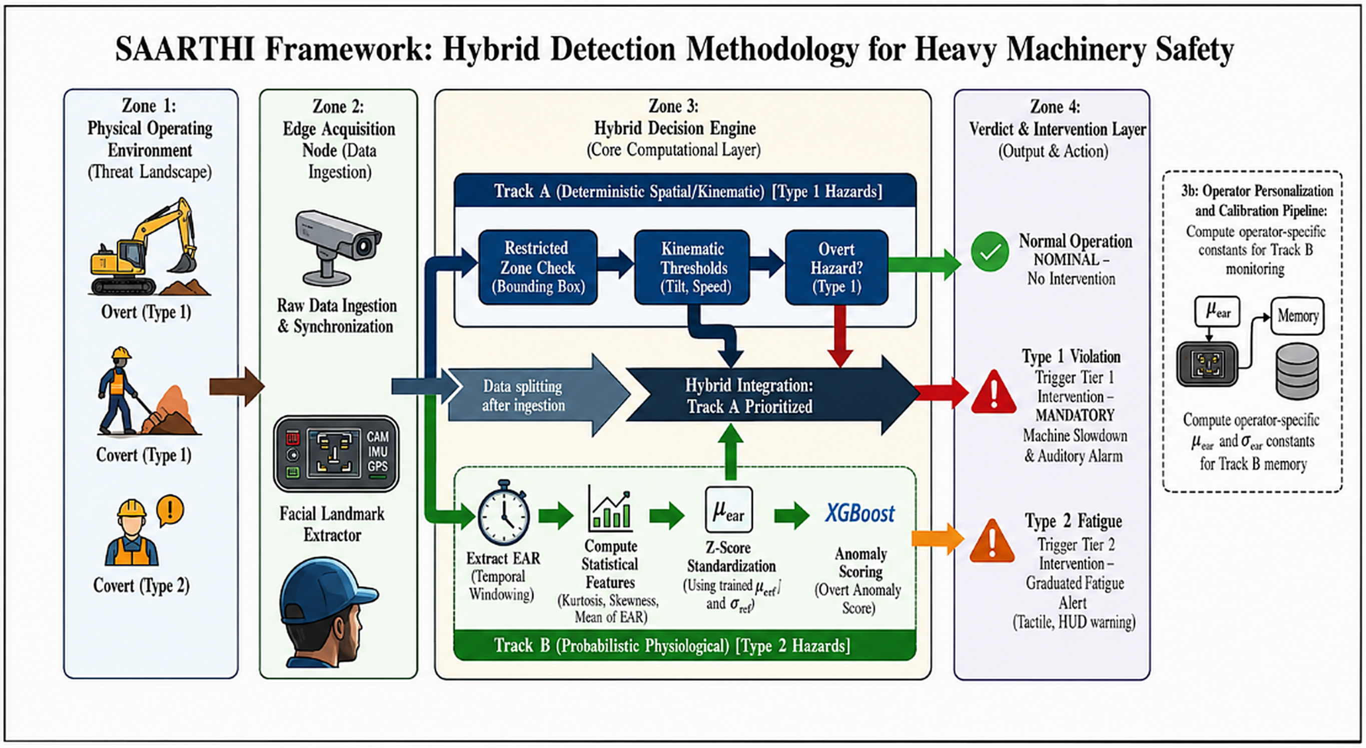

Fig. 1. *SAARTHI framework for hybrid detection methodology in heavy machinery safety. The framework integrates deterministic spatial/kinematic analysis (Track A) with probabilistic physiological monitoring (Track B) to detect and classify hazards into Type 1 (overt) and Type 2 (covert) categories, followed by appropriate intervention strategies.*

Fig. 1. SAARTHI framework for hybrid detection methodology in heavy machinery safety. The framework integrates deterministic spatial/kinematic analysis (Track A) with probabilistic physiological monitoring (Track B) to detect and classify hazards into Type 1 (overt) and Type 2 (covert) categories, followed by appropriate intervention strategies.

3) Window Size Selection and Stability Criterion: Choosing the window size w is a metrological trade-off between detection latency and statistical stability. Empirically, raw detection accuracy peaks at smaller windows — but that peak sits squarely in the white-noise-dominated regime, where frame-to-frame variance (from natural rapid blinks, for example) runs high. Short windows may look accurate on paper, yet they invite false positives driven by transient physiological noise.

To put the choice on firmer ground, we ran an Allan deviation analysis over steady-state operator streams. The noise floor bottomed out at an averaging time of $\tau_{opt} \approx 30.0$ samples — the stability plateau, where white noise is already minimized but random-walk drift (posture shifts and the like) has not yet taken over. SAARTHI therefore uses $w = 30$, sitting right at τ_{opt} and trading the marginal sensitivity of smaller windows for operational stability. At this size the total data-collection time stays under 1.5 seconds, comfortably inside the T_{hazard_onset} pre-intervention budget.

4) Z-Score Standardization: XGBoost's decision space is distance-sensitive, and our feature space is heterogeneous, so the features have to be scaled. Left unscaled, high-magnitude variables like variance would swamp low-range ones like skewness and tilt the decision boundary. The system applies Z-score standardization in real time:

$$z_i = \frac{x_i - \mu_{cal}}{\sigma_{cal}} \tag{6}$$

The constants μ_{cal} and σ_{cal} are fixed during the initial 30-second operator calibration and kept in memory, which spares the device from recomputing global statistics on every frame. Algorithm 2 spells out the full extraction logic.

B. Track A: Deterministic Spatial and Kinematic Verification

Track A is a fast, deterministic pre-filter that runs in $O(1)$. Its job is to catch Type 1 overt hazards the instant they occur — a pedestrian entering a restricted zone, an unsafe machine tilt — with no machine-learning overhead at all. It does this by testing the current state against a deterministic constraint set C_{rules}, implemented as a lookup over predefined spatial boundaries and kinematic thresholds.

Bounding-box intersection tests and threshold comparisons cost next to nothing on any edge processor, so this gate is cheap. Only states that clear Track A — those with no deterministic hazard — go on to Track B for the deeper physiological analysis.

Algorithm 1: Adaptive Rolling Window Data Ingestion

Input: Raw video stream S_V, telemetry stream S_T, window size w
Output: Temporal buffer B_t
Globals: Static ring buffer R of size w, head pointer p

```
while System Active do
    f_raw ← ReadFrame(S_V);
    t_raw ← ReadTelemetry(S_T);
    // Stage 1: detection integrity filtering
    if DetectFace(f_raw) = FAIL then
        continue; // discard frames lacking a clear face
    // Stage 2: extraction
    e_ear ← ExtractEAR(f_raw);
    k_state ← ExtractKinematics(t_raw);
    // Stage 3: buffer injection
    R[p] ← { e_ear, k_state, timestamp };
    p ← (p + 1) mod w;
    // check window saturation
    if p = 0 or BUFFERFULLFLAG then
        B_t ← Linearize(R);
        Trigger FeatureExtraction(B_t);
```

C. Track B: Statistical Physiological Anomaly Scoring (XGBoost)

Track B takes on the Type 2 covert hazard, the case where the operator still looks awake and engaged while the underlying physiological signal quietly drifts toward fatigue. To catch that drift, the system uses a gradient-boosted ensemble (XGBoost) trained on the standardized feature vector $\mathbf{z}$.

Most classifiers need labelled "fatigue" examples, which are expensive or simply unavailable for every individual operator. SAARTHI sidesteps that: the calibration phase learns a personalized envelope of normal operation for each operator, and any feature vector that strays well outside that envelope is flagged as a fatigue anomaly.

1) The Primal and Dual Formulations: The ensemble's goal is to carve a decision boundary in the feature space $\Phi(\mathbf{x})$ that separates the bulk of the training data — alert operator states — from the anomalous region. It does so by learning an additive ensemble of T weak learners that minimizes a regularized objective:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(f_t) \tag{7}$$

where l is the loss, $\hat{y}_i$ the predicted score, and $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ a penalty on tree complexity. SAARTHI sets $\lambda = 1.0$ for strict regularization, treating the outermost 1% of training samples as candidate edge-case fatigue anomalies.

2) The Gradient Boosting Kernel: A linear boundary cannot capture the tangled, non-linear interplay between higher-order moments like kurtosis and skewness. To build a surface expressive enough to resolve them, XGBoost grows trees over the feature space $\Phi(\mathbf{x})$, in effect approximating a kernel mapping. At inference time the decision function is:

$$D(\mathbf{z}) = \operatorname{sgn}\left(\sum_{t=1}^{T} \eta \cdot f_t(\mathbf{z}) - \rho\right) \tag{8}$$

where η is the learning rate and ρ the decision threshold. The regularizer's γ sets the minimum gain a split must earn, which in turn governs how sharply the boundary curves. Algorithm 3 outlines the offline procedure that produces the boosted model shipped to the edge device.

Algorithm 2: Statistical Feature Extraction

Input: Temporal buffer B_t, calibration stats $(\mu_{cal}, \sigma_{cal})$
Output: Normalized feature vector $\mathbf{z}$

```
// Compute raw mean (first pass)
μ ← 0;
for i ← 1 to w do
    μ ← μ + B_t[i].ear;
μ ← μ/w;
// Compute higher-order moments (second pass)
sum_sq ← 0;   sum_cu ← 0;   sum_qd ← 0;
for i ← 1 to w do
    diff ← B_t[i].ear − μ;
    sum_sq ← sum_sq + diff^2;
    sum_cu ← sum_cu + diff^3;
    sum_qd ← sum_qd + diff^4;
σ ← sqrt(sum_sq/(w − 1));
E_roc ← B_t[w].ear − B_t[w − 5].ear;
S_k ← (sum_cu/w)/σ^3;
K_u ← ((sum_qd/w)/σ^4) − 3;        // excess kurtosis
// Construct and standardize
x ← [ μ, σ, E_roc, S_k, K_u ];
for j ← 1 to 5 do
    z[j] ← (x[j] − μ_cal[j])/σ_cal[j];
return z;
```

Algorithm 3: Offline Training Procedure

Input: Training dataset X_{train}, params $(\eta, \lambda, \gamma, T)$
Output: Boosted model M, offset ρ

```
// Feature matrix computation
Z ← ExtractAndStandardize(X_train);
// Ensemble training
M ← XGBoost.Train(Z, params(η, λ, γ, T));
// Extract decision threshold
ρ ← ComputeThreshold(M, X_train);
// Model compression for edge
M ← Quantize(M, ρ);
return M;
```

When $D(\mathbf{z}) = -1$, the feature vector lies outside the calibrated operator's learned envelope, and a fatigue alert is raised.

3) Hybrid Logic Integration: At the heart of Zone 3, the final decision logic fuses the deterministic certainty of Track A with the probabilistic rigour of Track B; Algorithm 4 lays out the combined flow. Running the lightweight Track A check first keeps the heavier XGBoost inference of Track B on a low duty cycle.

The final safety verdict is recovered through the ensemble's decision function in Equation (8). If $D(\mathbf{z}) = -1$, the signal has fallen outside the calibrated operator's normal distribution, and a Tier-2 fatigue intervention fires.

Algorithm 4: Hybrid Execution Flow

```
Input: Feature vector z, kinematic state k_state, rules
       C_rules, model M, threshold ρ
Output: Safety verdict V, intervention tier
// Track A: deterministic gate
if CheckConstraints(k_state, C_rules) = VIOLATION
 then
    V ← HAZARD_TYPE_1;
    Trigger Tier1Intervention();
    return V;
// Track B: physiological anomaly
   scoring
score ← Σ_{t=1}^{T} η · f_t(z);
if sgn(score − ρ) = −1 then
    V ← HAZARD_TYPE_2;
    Trigger Tier2Intervention();
else
    V ← NOMINAL;
return V;
```

D. Tiered Intervention Protocol

When either track returns a positive verdict, SAARTHI responds with a structured, tiered protocol rather than a single binary alarm. The graduation matters in a heavy-machinery setting: a sudden, jarring alert in the middle of a delicate manoeuvre — a precise crane lift, say — could itself cause the accident it was meant to prevent.

Tier 1 — Deterministic override (Track A). Fires the instant a kinematic violation is confirmed — for example $\theta > \theta_{crit}$ while $v \neq 0$, or a bounding-box intrusion into a restricted zone. The response is non-negotiable: a mandatory speed-reduction command goes straight to the CAN bus and a loud cabin alarm sounds. This tier is independent of operator behaviour or any machine-learning state.

Tier 2 — Adaptive fatigue alert (Track B). Fires when $D(\mathbf{z}) = -1$, signalling a sustained drift outside the operator's normal envelope. Here the response builds gradually: first a tactile seat-vibration pulse, then a visual HUD warning if the anomalous state persists beyond δ_t consecutive windows. Should the operator not acknowledge within a preset timeout T_{ack}, the system escalates to a Tier-1-equivalent mandatory slow-down.

This restrictive admission-control approach keeps interruptions rare during normal operation while guaranteeing that genuine hazards always draw an immediate, deterministic response — which, in practice, is what earns operator trust and keeps the system in use.

E. Operator Personalization and Calibration

Population-level fatigue models share one fundamental weakness: physiology varies from operator to operator. A naturally narrow-eyed operator's baseline EAR mean μ_{alert} can look, statistically, like another operator's fatigued state — and the resulting stream of false positives steadily erodes trust in the system.

SAARTHI answers this with a mandatory 30-second calibration at the start of every shift. The operator is asked to look ahead, relaxed and alert, while the system ingests N_{cal} frames and computes operator-specific statistics:

$$\mu_{cal} = \frac{1}{N_{cal}} \sum_{i=1}^{N_{cal}} e_i, \qquad \sigma_{cal} = \sqrt{\frac{1}{N_{cal}-1} \sum_{i=1}^{N_{cal}} (e_i - \mu_{cal})^2} \tag{9}$$

These constants then Z-score-normalize every incoming feature vector (Equation 6), anchoring the whole pipeline to the individual's physiology rather than a generic population mean. Each profile is saved to the operator profile store, so returning operators skip re-calibration — though the profile can still be refreshed over time to track long-term physiological drift.

V. Conclusion and Future Work

This paper introduced SAARTHI, a personalized AI safety copilot for heavy-machinery operators facing multi-hazard industrial risks. Moving away from the siloed, rule-based alarms that dominate the field, the system fuses hazard detection into a single Hybrid Decision Engine governed by a constrained Reinforcement Learning policy—an approach that tries to reconcile consistent safety enforcement with the fact that no two operators have quite the same needs.

A. Summary of Contributions

Three things stand out from the design and integration of the system.

- **Multi-modal risk fusion.** Combining fatigue, blind-spot, tilt, and contextual risk signals into one real-time safety score captures compound hazards that any single detector would miss on its own. The individual detectors supply the sensory groundwork, but it's the fusion logic in the Hybrid Decision Engine that actually decides when intervention is warranted.
- **Balancing safety and personalization.** The PPO-based policy weighs predefined safety constraints against operator-specific delivery preferences, choosing interventions based on accessibility needs, experience level, and stated alert preferences. That's a meaningfully different

approach from the rigid, uniform alarm thresholds most systems still rely on, and it shows that transparent, explainable safety copilots are achievable without giving up consistency.

- **Scalable and accessible deployment.** Centralizing personalization in operator profiles, and surfacing decisions through a supervisor dashboard, gives SAARTHI an edge over fragmented, single-purpose safety hardware—it scales across construction, mining, and manufacturing fleets while closing an accessibility gap those single-purpose devices never addressed. The Explainable AI layer on top of that keeps both operators and supervisors able to trust why any given intervention happened.

B. Limitations and Future Work

The framework is a solid starting point, but the design process surfaced a few areas that clearly need more work.

- **Automatic operator identification.** Right now the system assumes an operator manually selects their profile at shift start—accessibility settings, preferred language, alert preferences, all of it. On a busy or rotating crew, that step gets skipped often enough that the cabin defaults to generic settings that don't actually match whoever's in the seat. Facial recognition for automatic operator identification is the obvious fix: detect who's in the cabin, load their profile—including whatever language settings they've already configured—without requiring a manual login. It closes the gap between what the system is meant to do and what actually happens in practice.
- **Wearable sensor fusion for fatigue robustness.** The fatigue model currently leans almost entirely on facial-landmark tracking, which struggles under poor cabin lighting, camera occlusion, or when an operator is wearing a dust mask or safety goggles. Bringing in wearable physiological sensors—heart-rate variability bands, EEG headsets—to fuse with the existing vision-based estimate should make the fatigue score noticeably more robust to exactly those failure modes.
- **Field validation and policy hardening.** So far the reinforcement learning policy has only been trained and tested in simulated, controlled cabin conditions. Real deployment across construction, mining, and manufacturing sites will expose it to dust, vibration, and lighting variability that simulation just doesn't capture well, and that gap needs to be closed before the policy can be trusted at scale. The plan is a supervised Field Calibration phase—collecting real-world incident and near-miss data to refine the XGBoost context model's risk thresholds and retrain the PPO policy under actual operating conditions, while keeping the predefined safety constraints intact throughout.